Lexique

Les termes techniques du cours, en une phrase, avec leur équivalent anglais

Kevin Bardoux

Table des matières

A 1
B 2
C 2
D 2
E 2
F 3
G 3
I 3
L 3
M 3
N 4
P 4
R 4
S 5
T 5
V 5

Chaque terme est défini en une phrase, avec son équivalent anglais entre parenthèses et la semaine où il apparaît. Le lexique grandit au fil de la formation : un terme y entre avec sa semaine.

A

Algorithme (*algorithm*) : Suite d'instructions précises, exécutées dans l'ordre, qui résout un problème. *Semaine 1.*

Argument (*argument*) : Valeur donnée à un programme ou à une fonction au moment de l'appel. *Semaine 2*.

B

Binaire (*binary*) : Façon d'écrire les nombres avec deux chiffres seulement, 0 et 1. *Semaine 2*.

Bit (*bit*) : Chiffre binaire : 0 ou 1, la plus petite information que manipule un ordinateur. *Semaine 2*.

Bogue (*bug*) : Erreur dans un programme, qui le fait planter ou donner un résultat faux. *Semaine 1*.

Booléen (*boolean*) : Valeur qui ne peut être que `true` (vrai) ou `false` (faux). *Semaine 2*.

Boucle (*loop*) : Instruction qui répète un groupe d'instructions tant qu'une condition est vraie ou un nombre de fois donné. *Semaine 1*.

C

Chaîne de caractères (*string*) : Valeur de type texte, écrite entre guillemets. *Semaine 2*.

Commentaire (*comment*) : Texte du code ignoré par la machine, écrit pour les humains (après `//`). *Semaine 2*.

Condition (*condition*) : Question dont la réponse est vraie ou fausse, qui décide si des instructions s'exécutent. *Semaine 1*.

Constante (*constant*) : Variable déclarée avec `const` : son nom désigne toujours la même valeur. *Semaine 2*.

D

Débogage (*debugging*) : Démarche qui trouve et corrige la cause d'un comportement faux : lire, isoler, tester, corriger. *Semaine 3*.

Disque (*disk, drive*) : Support qui garde les fichiers quand l'ordinateur est éteint. *Semaine 2*.

E

ECF (—) : Évaluation en cours de formation : une épreuve qui mesure des compétences du titre. *Semaine 3*.

Éditeur de code (*code editor*) : Logiciel pour écrire et enregistrer des fichiers de programmation en texte brut (VS Code dans la formation). *Semaine 2*.

Égalité stricte (*strict equality*) : Comparaison `===` : vraie si les deux valeurs ont le même type et la même valeur. *Semaine 2*.

ESLint (*ESLint*) : Outil qui relit le code sans le lancer et signale chaque écart à une règle. *Semaine 2*.

Événement (*event*) : Ce qui se produit (un clic, une touche, le drapeau vert) et qui déclenche un script. *Semaine 1*.

Extension de fichier (*file extension*) : Fin du nom d'un fichier, après le point, qui indique son type (`.js`, `.txt`). *Semaine 2*.

F

Fonction (*function*) : Bloc d'instructions nommé, avec des paramètres, qui renvoie un résultat quand on l'appelle. *Semaine 3*.

G

Grand O (*Big O*) : Notation qui dit comment le nombre d'étapes d'un algorithme grandit quand les données grandissent. *Semaine 3*.

I

Index (tableau) (*index*) : Numéro de position d'une valeur dans un tableau ou d'un caractère dans une chaîne, à partir de `0`. *Semaine 3*.

L

Littéral de gabarit (*template literal*) : Texte écrit entre accents graves, dans lequel `${…}` insère une valeur. *Semaine 2*.

LTS (*Long Term Support*) : Version d'un logiciel maintenue longtemps, recommandée pour un usage sérieux. *Semaine 2*.

M

Mémoire vive (*RAM*) : Mémoire qui contient les programmes en cours et leurs données ; elle s'efface quand l'ordinateur s'éteint. *Semaine 2*.

Message (Scratch) (*broadcast*) : Signal nommé envoyé par un sprite, qui démarre les scripts des sprites qui l'attendent. *Semaine 2*.

Méthode (*method*) : Fonction attachée à une valeur, appelée avec un point : `texte.toUpperCase()`. *Semaine 3*.

N

NaN (*Not a Number*) : Valeur obtenue quand une conversion en nombre échoue (`Number('abc')`). *Semaine 2*.

Node.js (*Node.js*) : Programme qui exécute du JavaScript en dehors du navigateur, dans un terminal. *Semaine 2*.

npm (*npm*) : Gestionnaire de paquets de Node.js : il télécharge et installe les paquets d'un projet. *Semaine 2*.

P

Paquet (*package*) : Code écrit par d'autres, publié sur npm, qu'on installe dans un projet. *Semaine 2*.

Paramètre (*parameter*) : Nom, dans la déclaration d'une fonction, qui reçoit une valeur à chaque appel. *Semaine 3*.

Portée (*scope*) : Partie du programme où une variable est visible et utilisable. *Semaine 3*.

Processeur (*processor, CPU*) : Composant qui exécute les instructions d'un programme, une à la fois. *Semaine 2*.

Pseudo-code (*pseudocode*) : Algorithme écrit en français structuré, une instruction par ligne, avant le vrai code. *Semaine 1*.

R

Recherche dichotomique (*binary search*) : Recherche dans une liste triée qui regarde le milieu et ne garde qu'une moitié, à chaque étape. *Semaine 3*.

Recherche linéaire (*linear search*) : Recherche qui regarde les cases une par une, du début à la fin. *Semaine 3*.

Renvoyer (`return`) (*return*) : Instruction qui termine une fonction et indique la valeur qu'elle renvoie. *Semaine 3*.

REPL (*REPL*) : Console interactive de Node : on tape une ligne, Node l'exécute et affiche le résultat. *Semaine 2*.

Reste (opérateur) (*remainder, modulo*) : Opérateur `%` : le reste de la division entière de deux nombres. *Semaine 2*.

S

Sprite (*sprite*) : Personnage ou objet animé par un programme Scratch. *Semaine 1*.

Style défensif (*defensive programming*) : Façon de coder qui traite d'abord les entrées invalides, avant le cas normal. *Semaine 2*.

T

Tableau (*array*) : Liste ordonnée de valeurs, repérées par un index qui commence à `0`. *Semaine 3*.

Terminal (*terminal, shell*) : Programme où l'on tape des commandes, qu'il exécute dans un dossier. *Semaine 2*.

Test (*test*) : Petit programme qui lance le vôtre et compare le résultat obtenu au résultat attendu. *Semaine 2*.

Tri (*sort*) : Rangement des valeurs d'une liste dans l'ordre (par sélection, à bulles…). *Semaine 3*.

Type (*type*) : Sorte d'une valeur : nombre, texte, vrai ou faux… *Semaine 2*.

V

Variable (*variable*) : Nom qui désigne une valeur, pour la retrouver et la modifier plus loin dans le programme. *Semaine 1*.